Table of content

Preamble

The present document gives some informations upon the HDL code available in the 01 Bitbucket directory. The code may not be optimal but it passes the tests supplied in the Nand2Tetris project files at least.

`Not` chip

Truth tables :

a	not a
0	1
1	0

a	a	a nand a
0	0	1
1	1	0

So : `not(a) = a nand a`

And in HDL : `Nand(a=in, b=in, out=out);`

`And` chip

Since `a nand b = not(a and b)`, `a and b = not(a nand b)`. So `a and b = (a nand b) nand (a nand b)` based on the not chip.

In HDL, the result of `a nand b` is stored into `n1` and then we compute `n1 nand n1` :

```
Nand(a=a, b=b, out=n1);
Nand(a=n1, b=n1, out=out);
```

`Or` chip

Truth tables :

a	b	a or b	not(a)	not(b)	not(a) and not(b)	not(not(a) and not(b))
0	0	0	1	1	1	0
0	1	1	1	0	0	1
1	0	1	0	1	0	1
1	1	1	0	0	0	1

So : `a or b = not(not(a) and not(b)) = not(a) nand not(b)`

In HDL :

```
Nand(a=a, b=a, out=nota);
Nand(a=b, b=b, out=notb);
Nand(a=nota, b=notb, out=out);
```

`Xor` chip

The `Xor.hdl` file contains 2 versions. The first one uses only `Nand` gates. The second one (commented in the code) uses `Not`, `And` and `Or` gates. The translation can be seen through the following truth table :

a	b	not(a)	not(b)	not(a) and b	a and not(b)	(not(a) and b) or (a and not(b))
0	0	1	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

And also through the fact that `a xor b` can be seen as `(a or b) and not(a and b)`. Which can be expressed as `(a and not(b)) or (b and not(a))`. I don't know if there is a simpler have or or a more elegant way to express the chip.

`Mux` chip

From the definition of the chip, `mux(a, b, sel) = (a and not(sel)) or (b and sel)` which can be translated as `(a nand (sel nand sel)) nand (b nand sel)`. And can be verified through the following truth table :

a	b	sel	sel nand sel	a nand (sel nand sel)	b nand sel	(a nand (sel nand sel)) nand (b nand sel)
0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	0	1	0	1	1
1	1	0	1	0	1	1
0	0	1	0	1	1	0
0	1	1	0	1	0	1
1	0	1	0	1	1	0
1	1	1	0	1	0	1

`DMux` chip

From the definition of the chip, `dmux(in, sel) = (in, 0) if sel = 0 or (0, in) if sel = 1`
Which has been translated like this in `Dmux.hdl` :

```
// a = in And Not(sel)
// b = in And sel
Not(in=sel, out=nsel);
And(a=in, b=nsel, out=a);
And(a=in, b=sel, out=b);
```

`Not16` chip

From the definition of the chip, we have `in[16]` as input, `out[16]` as output and `out[i] = Not(in[i])` for `i` in `[0, 15]`. Which can give the following HDL code :

```
Not(in=in[0], out=out[0]);
Not(in=in[1], out=out[1]);
(...)
Not(in=in[15], out=out[15]);
```

The following command line for loops can generate the previous code automatically :

- Windows : `for /L %N in (0, 1, 15) do @echo Not(in=in[%N], out=out[%N]);`
- Bash : `for N in {0..15}; do echo "Not(in=in[${N}], out=out[${N}]);" ; done`

`And16` chip

Straightforward implementation from the previous chip.

`Or16` chip

Likewise.

`Mux16` chip

Likewise.

`Or8Way` chip

Since `out=Or(in[0],in[1],...,in[7])`, we first compute `or1 = Or(in[0], in[1])` and then `or2 = Or(or1, in[2])` and so on until `out = Or(or6, in[7])`.

`Mux4Way16` chip

Work in progress...

`Mux8Way16` chip

Work in progress...

`DMux4Way` chip

Work in progress...

`DMux8Way` chip

Work in progress...

to be continued